1. Problem Statement, Objectives, and Assumptions

Problem Statement

Managing multi-subject academic scores for a group of students, calculating individual totals and averages, identifying class toppers, and computing subject-wise averages manually can be tedious and prone to calculation errors. An automated command-line tool is required to process student score matrices efficiently.

Objectives

* Prompt for the number of students and collect individual scores across fixed subjects (`Python`, `Maths`, and `English`).

* Store student marks in a nested list (matrix) structure.

* Compute total and average marks for each student.

* Identify and display the top-performing student based on total scores.

* Calculate and display the average score for each individual subject across all students.

Assumptions

* The curriculum consists of three predefined subjects: Python, Maths, and English.

* Scores are entered as valid numerical values by the user.

---

2. Algorithm, Flowchart, and Pseudocode

Algorithm

1. Prompt the user for the number of students (`n`).

2. Define a list of `subjects` containing `["Python", "Maths", "English"]`.

3. Run a loop `n` times to collect each student's name and their scores for each subject, appending them to `names` and `marks` lists.

4. Initialize an empty `totals` list.

5. Iterate through each student to compute their `total` score and `average` percentage, and print their individual report.

6. If `n > 0`, find the index of the maximum total score to identify the top student and print their details.

7. Iterate through each subject column to calculate the aggregate `subject_total` across all students and print the average score for each subject.

Flowchart Representation

```text
[Start] --> [Input Number of Students n] --> [Define Subjects: Python, Maths, English]
    --> [Loop n times: Input Name & Subject Scores]
        --> [Store in names and marks matrix]
    --> [Loop through students: Calculate individual total and average]
    --> [Print Student Report]
    --> <Condition: n > 0?>
        |---> [True] --> [Identify and Print Top Student]
        |---> [False] --> [Skip]
    --> [Loop through subjects: Calculate subject-wise averages]
    --> [Print Subject Averages] --> [End]

```

Pseudocode

```text
BEGIN
  PRINT "Number of students: "
  READ n
  SET subjects = ["Python", "Maths", "English"]
  SET names = []
  SET marks = []

  FOR i FROM 0 TO n-1 DO
    PRINT "Student name: "
    READ name
    SET row = []
    FOR EACH subject IN subjects DO
      PRINT name + " - " + subject + " marks: "
      READ score
      APPEND score TO row
    END FOR
    APPEND name TO names
    APPEND row TO marks
  END FOR

  SET totals = []
  PRINT "\n--- STUDENT REPORT ---"
  FOR i FROM 0 TO n-1 DO
    SET total = SUM(marks[i])
```

```
    SET average = total / LENGTH(subjects)
    APPEND total TO totals
    PRINT names[i], "Marks:", marks[i], "Total:", total, "Average:", average
  END FOR

  IF n > 0 THEN
    SET top = INDEX OF MAX(totals)
    PRINT "Top student:", names[top], "with total", totals[top]
  END IF

  FOR j FROM 0 TO LENGTH(subjects)-1 DO
    SET subject_total = 0
    FOR i FROM 0 TO n-1 DO
      subject_total = subject_total + marks[i][j]
    END FOR
    IF n > 0 THEN
      PRINT subjects[j], "average:", subject_total / n
    END IF
  END FOR
END
```

---

3. Python Source Code and Test Cases

Python Source Code

```python
names = []
marks = []
n = int(input("Number of students: "))
subjects = ["Python", "Maths", "English"]

for i in range(n):
  name = input("Student name: ")
  row = []
  for subject in subjects:
    score = float(input(name + " - " + subject + " marks: "))
    row.append(score)
  names.append(name)
  marks.append(row)

totals = []
print("\n--- STUDENT REPORT ---")
for i in range(n):
  total = sum(marks[i])
  average = total / len(subjects)
  totals.append(total)
  print(names[i], "Marks:", marks[i],
      "Total:", total, "Average:", round(average, 2))

if n > 0:
  top = totals.index(max(totals))
  print("Top student:", names[top], "with total", totals[top])
```

```
for j in range(len(subjects)):
    subject_total = 0
    for i in range(n):
        subject_total += marks[i][j]
    if n > 0:
        print(subjects[j], "average:", round(subject_total / n, 2))
```

(Source code adapted from).

Test Cases & Sample Output

Test Case ID	Scenario Description	User Inputs	Expected Output / Behavior
TC-01	Multi-Student Batch Analysis	Students: `2`	

1: `Aarav`, Python: `85`, Maths: `90`, English: `80`

2: `Diya`, Python: `92`, Maths: `88`, English: `95` | Generates individual reports, crowns Diya as the top student, and prints subject averages. |

Simulated Terminal Output (for TC-01)

```text
Number of students: 2
Student name: Aarav
Aarav - Python marks: 85
Aarav - Maths marks: 90
Aarav - English marks: 80
Student name: Diya
Diya - Python marks: 92
Diya - Maths marks: 88
Diya - English marks: 95

--- STUDENT REPORT ---
Aarav Marks: [85.0, 90.0, 80.0] Total: 255.0 Average: 85.0
Diya Marks: [92.0, 88.0, 95.0] Total: 275.0 Average: 91.67
Top student: Diya with total 275.0
Python average: 88.5
Maths average: 89.0
English average: 87.5
```

---

4. Conclusion

The student marks matrix analysis script efficiently handles multi-dimensional academic data using nested lists and loops. It automates student aggregation, identifies top performers, and computes subject-wise averages accurately, offering a complete assessment utility for educators.